

Problem Set 5

CMSC 27200: Theory of Algorithms, Winter'25

Assigned February 16, Due February 24

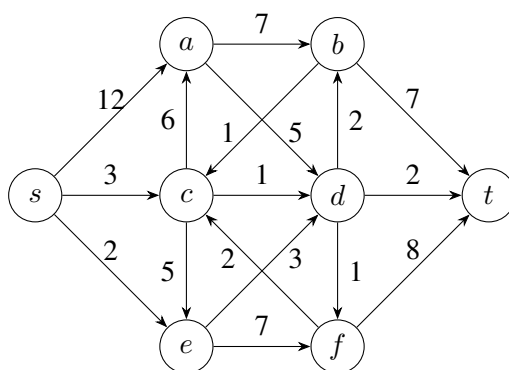
Problem 0: Collaborators and Outside Sources

Who did you collaborate with?

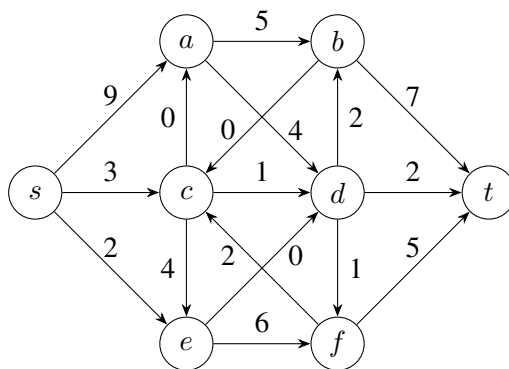
Did you use any outside sources? If so, which sources did you use?

Problem 1: Ford-Fulkerson

Graph:



Current flow:



Starting from the given flow, use the Ford-Fulkerson algorithm to find a maximum flow from s to t and find a minimum cut proving that this flow is a maximum flow. For each iteration (including the iteration where the algorithm terminates), show the residual graph and the flow path that you find from s to t (if such a path exists).

Note: If an edge in the residual graph has capacity 0, showing this edge is optional (showing the edge or not showing the edge are both fine).

Problem 2: Weighted Interval Scheduling with Two Workers

Let's say that there are n jobs where job i has an interval $[a_i, b_i]$ and a weight w_i , where the interval i has weight w_i . Each of you can work on one job at a time (you must finish your current job before starting a new job). Which jobs should you and your partner take in order to maximize the total weight of the completed jobs?

We can state this problem mathematically as follows. Given n intervals $\{[a_i, b_i] : i \in [n]\}$ and weights $\{w_i, i \in [n]\}$, choose schedules $S = (i_1, \dots, i_m)$ and $S' = (i'_1, \dots, i'_{m'})$ such that

1. For each $j \in [m - 1]$, $a_{i_{j+1}} \geq b_{i_j}$.
2. For each $j \in [m' - 1]$, $a_{i'_{j+1}} \geq b_{i'_j}$.
3. $\{i_1, \dots, i_m\} \cap \{i'_1, \dots, i'_{m'}\} = \emptyset$ (no job is taken by both workers).

and the total weight $\sum_{j=1}^m w_{i_j} + \sum_{j=1}^{m'} w_{i'_j}$ of the completed jobs is maximized.

- (a) Give a polynomial time algorithm to find schedules for you and your partner which maximizes the total weight of the completed jobs.
- (b) Briefly explain why your algorithm is correct. You do not need a rigorous proof.
- (c) What is the runtime of your algorithm?

Note: This problem will be graded according to the rubric for dynamic programming. An exemplary grade can be earned for part a.

Problem 3: Ford-Fulkerson With No Backwards Edges (Rephrasing of Problem 11 in Chapter 7 of Kleinberg-Tardos)

Your friend wonders, what happens if we don't have backwards edges in the Ford-Fulkerson algorithm? In other words, what happens if, after an iteration of Ford-Fulkerson where we find a path from s to t and route c_{min} flow through it (where c_{min} is the minimum remaining capacity of an edge on this path), we just reduce the capacities of the edges along this route by c_e in the residual graph (and don't increase the capacities of the edges going backwards by c_{min} as we are ignoring them)?

Your friend conjectures that the Ford-Fulkerson algorithm with no backwards edges will still always give a constant factor approximation, i.e., there exists a constant $c > 0$ such that if the

maximum flow is F , regardless of how we choose the paths from s to t we will obtain a total flow of at least cF from s to t . Is your friend's conjecture correct? Either prove your friend's conjecture or describe how to construct a counterexample for any $c > 0$. Hint is available.

Note: This problem will be graded according to True/False rubric. An exemplary grade can be earned for this problem.

Challenge question (not for credit, may be very hard): Your friend also conjectures that for any input graph G , there always exists a way for the Ford-Fulkerson algorithm to choose the paths from s to t such that no backwards edges are needed. Is this conjecture true?

Problem 4: Routing Data Packets

In this problem, you are asked to route as many data packets as you can through in a network. Formally, we are given a directed graph $G = (V, E)$, a set of source nodes $S = \{s_1, \dots, s_k\} \subseteq V$, and a set of sink nodes $T = \{t_1, \dots, t_\ell\} \subseteq V$. Each source node s_i holds an integer $n_i > 0$ number of data packets that need to be routed to one of the sink nodes and each sink node t_j has an integer capacity $m_j > 0$ which is the maximum number of data packets it can receive. Source nodes only have outgoing edges and sink nodes only have incoming edges. Every node v which is not a source node or sink node has a capacity c_v which is the maximum number of data packets that is allowed to pass through the node v .

- (a) Give a reduction from this problem to max flow. In other words, given an instance of this routing data packets problem, you should construct an instance of max flow such that an integer-valued flow of F from s to t is more or less equivalent to a way to route F packets from source nodes to sink nodes. Hint is available.
- (b) Given k paths from source nodes to sink nodes in the network such that
 1. For each source node s_i , at most n_i of the paths start at s_i .
 2. For each middle node v , at most c_v of the paths pass through v .
 3. For each sink node t_j , at most m_j paths end at t_j .

(i.e., a solution to the routing data packets problem which routes k packets from a source node to a sink node), how can you find a flow of k from s to t in the instance of max flow you constructed in part a?

Note: While this part is relatively simple, it gives a justification that your algorithm in part c will give an optimal solution to the routing data packets problem.

- (c) Given an integer-valued flow of F from s to t in the instance of max flow you constructed in part a, how can we use this flow to find F paths from a source node to a sink node in the network satisfying the conditions listed in part b (which we can use to route F data packets through the network)?
- (d) What is the runtime for constructing the instance of max flow, running Ford-Fulkerson to obtain an integer-valued max flow, and extracting paths for the data packets from this flow?

Note: Parts b and c will be graded together as your algorithm and the proof that your algorithm is correct. An exemplary grade can be earned for this.

Hints

3. What happens if your path from s to t goes back and forth across the minimum cut?
- 4a. Similar to the vertex-disjoint paths problem, you will want to enforce the capacity constraint on a middle node v by splitting it into two vertices.